

Sistemas operativos en misiones espaciales

Qué hace el sistema operativo cuando no puede fallar

Daniel Berumen Valdez

Sistemas Operativos · Grupo 7 · Semestre 2027-1

Facultad de Ingeniería, UNAM

EL PROBLEMA

Cuando no se puede presionar reiniciar

En tu computadora

- Si se traba, la reinicias
- Lo peor: pierdes unos minutos de trabajo
- Que algo tarde un segundo más casi no importa

En una nave en Marte

- Nadie puede apretar el botón de reinicio
- Una orden desde la Tierra tarda varios minutos
- Debe responder a tiempo y resistir la radiación



PREGUNTA GUÍA

¿Qué hace el sistema operativo cuando no puede fallar?

¿Qué hace el sistema operativo de una nave espacial cuando no puede fallar, y qué hace cuando falla de todos modos?



Cinco casos de estudio

1969

Apolo 11

Sobrecarga y reinicio controlado

1997

Mars Pathfinder

Inversión de prioridades

2006

CALIPSO

Radiación y barrido de memoria

2004

Spirit

Cuando reiniciar no basta

2021

Ingenuity

Linux en Marte

Tiempo real: el resultado debe llegar a tiempo

Tiempo real duro

Incumplir un plazo una sola vez puede provocar una falla grave

Ejemplo: Bolsa de aire de un automóvil

En una nave: Control de orientación, estabilidad de vuelo

Tiempo real suave

Un retraso ocasional empeora el resultado, pero no lo arruina

Ejemplo: Videollamada

En una nave: Envío de fotografías, registros

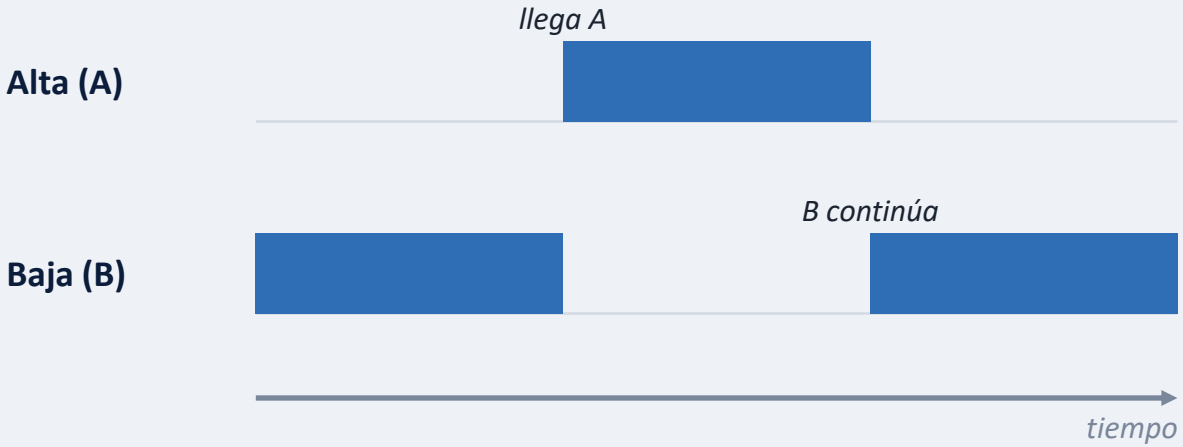


En tiempo real, llegar tarde equivale a equivocarse

Planificación por prioridades con apropiación

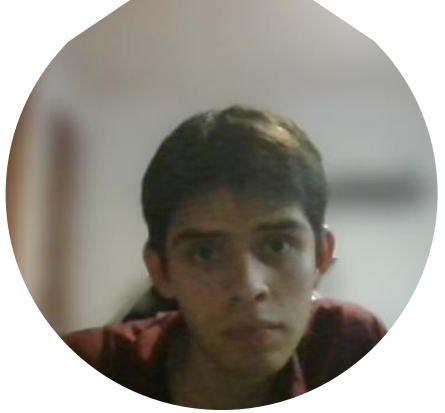
1. **Siempre corre la tarea lista de mayor prioridad**
   Las tareas críticas se atienden primero, sin esperar a las demás

2. **Con apropiación (preemptive)**
   Si llega una tarea más importante, desplaza de inmediato a la que corre

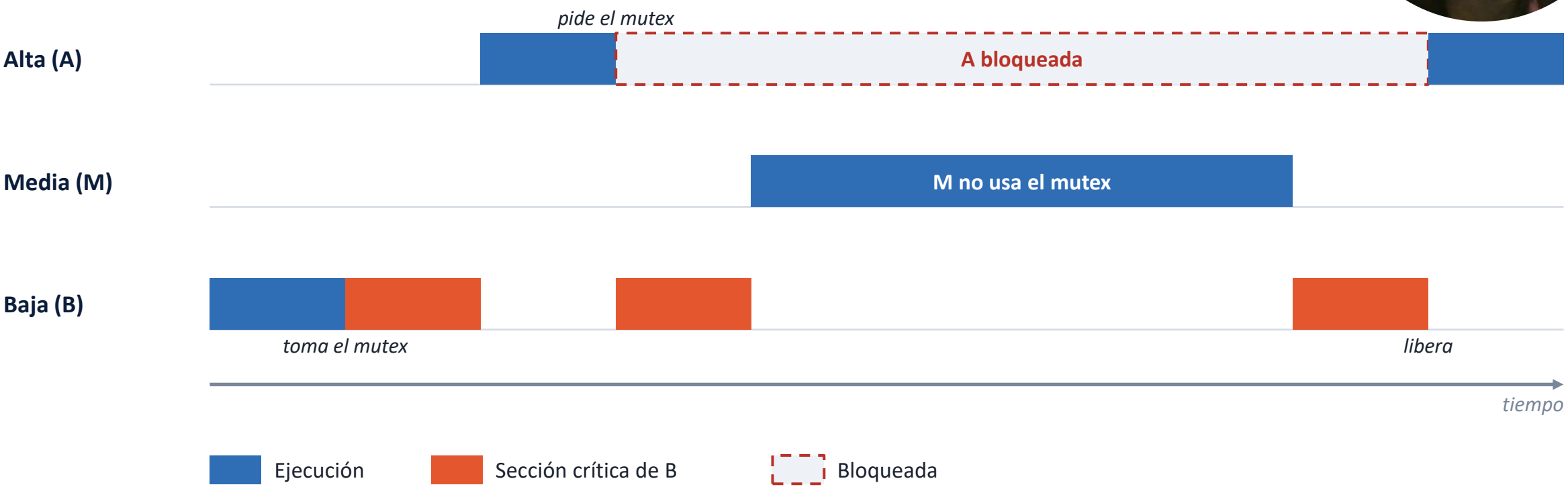


Es predecible... hasta que las tareas comparten recursos





Inversión de prioridades



Inversión no acotada: la tarea más importante espera a una de prioridad media que ni siquiera usa el recurso

Dos soluciones clásicas

Herencia de prioridad

Mientras B tenga un mutex que A necesita, B hereda temporalmente la prioridad de A.

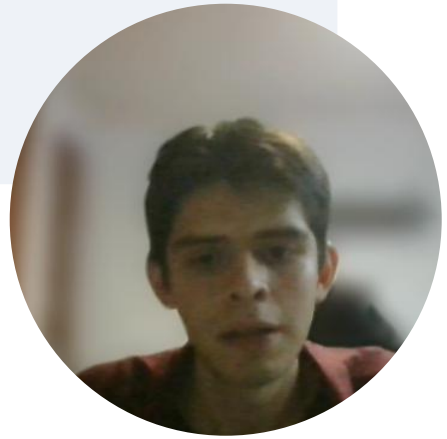
M ya no puede desplazarla: B termina, libera el mutex y recupera su prioridad.

Techo de prioridad

Cada mutex recibe de antemano la prioridad de la tarea más importante que pueda usarlo.

Más estricto: acota la espera a una sola sección crítica y evita bloqueos mutuos.

Formalizadas por Sha, Rajkumar y Lehoczky (1990). La idea del techo ya aparecía en el lenguaje Mesa (Lampson y Redell, 1980).



Radiación y watchdog



Single Event Upset (bit flip)

0	1	1	**1**	1	0	0	1

una partícula cambió este bit de 0 a 1

Hardware: ECC detecta y corrige al leer

Software: el barrido de memoria evita que los errores se acumulen

Watchdog

El software reinicia un temporizador periódicamente

↓

Si llega a cero: reinicio a un estado conocido

Como el pedal del hombre muerto de un tren: si el conductor deja de responder, el tren frena.

¿Por qué no usar un sistema operativo común?



Sistema de escritorio

Como un restaurante: intenta atender bien a todos en promedio, aunque alguno espere de más.

Sistema de tiempo real

Como urgencias: el más grave entra primero y se sabe cuánto esperará como máximo.

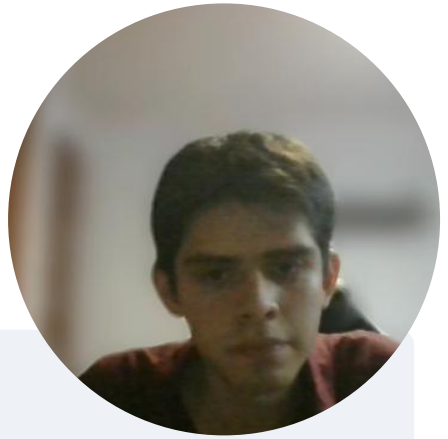
133 MHz

Frecuencia original del RAD750, procesador de Curiosity y Perseverance

A cambio: 1 error por radiación cada 10^{11} bits por día

Lo que se le pide al sistema operativo de una nave (cFS, NASA):
planificación por prioridades con apropiación, colas de mensajes, semáforos, interrupciones y sistema de archivos

Dos sistemas operativos de tiempo real



VxWorks

Comercial · Wind River

- 256 niveles de prioridad (0 es la más alta)
- Planificación por prioridades con apropiación
- Sistema operativo de todos los rovers de la NASA en Marte

Pathfinder 1997

Spirit / Opportunity 2004

Curiosity 2012

Perseverance 2021

RTEMS

Código abierto · desde 1988

- Muy usado por la ESA y la NASA (MMS, sonda solar Parker)
- Precalificado por la ESA, categoría B (2024); cada misión repite las pruebas
- **Se compila junto con la aplicación, sin separación entre espacio de usuario y núcleo**

core Flight System (cFS), NASA Goddard



Aplicaciones	Programas de cada misión
cFE	Servicios comunes
OSAL y PSP	Ocultan el SO y el hardware
RTOS	VxWorks, RTEMS
Hardware	Procesador y dispositivos

Cada capa solo conoce a la de abajo

El OSAL traduce una misma llamada al SO real

Sistema operativo	OS_TaskCreate se convierte en
VxWorks	`taskSpawn`
RTEMS	`rtems_task_create + rtems_task_start`
Linux (POSIX)	`pthread_create`

cFE para recuperarse: reinicia una aplicación o todo el procesador y conserva los datos críticos. Health and Safety vigila las aplicaciones y atiende el watchdog.

La Apollo Guidance Computer y su Executive

2048

palabras de memoria borrable (su RAM)

36 864

palabras de memoria fija

≈ 8

core sets: cupo máximo de trabajos registrados

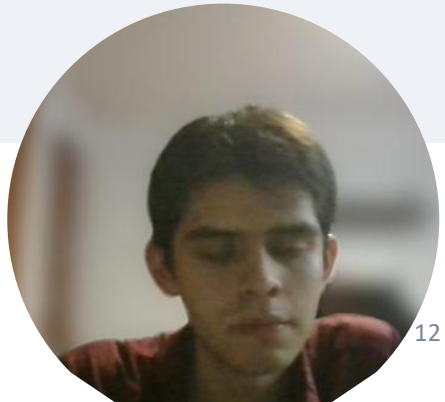
Executive

Administra los jobs por prioridad. Cada job ocupa un core set, el equivalente al bloque de control de proceso.

Waitlist

Dispara tasks muy cortas en momentos exactos mediante interrupciones del temporizador.

Las fuentes no coinciden en el número de core sets: Eyles (2004) indica ocho y Adler (1998), siete.



Las alarmas 1202 y 1201



1. Radar de encuentro encendido durante el descenso
2. Señales con la misma frecuencia, pero desfasadas
3. Miles de pulsos sin sentido roban ≈ 13 % del procesador
4. **SERVICER no termina y sus copias se acumulan**

10 a 15 %

tiempo libre que le quedaba a la AGC en el descenso

≈ 13 %

tiempo que robaban las interrupciones del radar

1202

Sin core set libre

1201

Sin VAC area libre

En términos del curso: la tabla de procesos se llenó porque el proceso de menor prioridad nunca alcanzaba a terminar.

Reiniciar sin perder el rumbo

1. Vacía la lista de jobs, incluidas las copias atrasadas de SERVICER
2. Reinicia solo los trabajos protegidos, desde su último punto de control
3. Lo no esencial simplemente desaparece

«We're Go on that alarm»

Steve Bales, con el respaldo de Jack Garman, decide continuar. Armstrong toma el control semimanual y el Eagle aterriza.

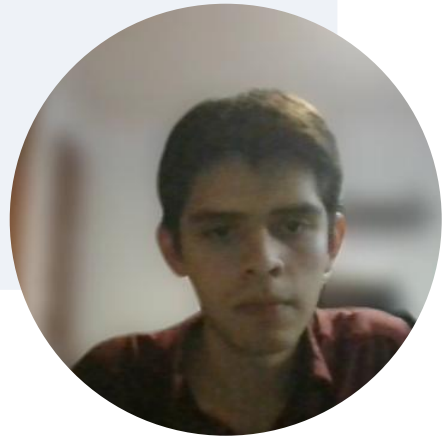
Precisiones

No fue un error de programación: era la respuesta prevista ante sobrecarga.

La causa se discute: Eyles culpa un documento de interfaz; Hamilton, un procedimiento de los astronautas.

Fue un trabajo colectivo: Laning diseñó el Executive y el equipo de Hamilton el software de vuelo.

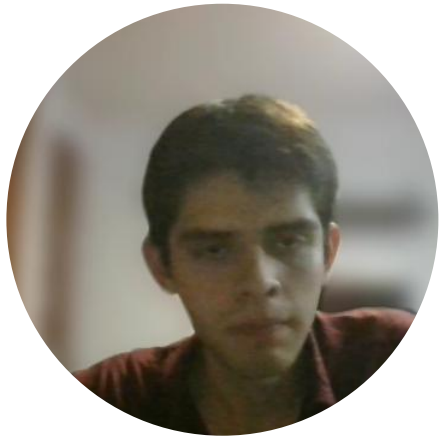
Dato curioso: la protección de reinicio se pensó para fallas de energía.



Las tareas involucradas

Tarea	Prioridad	Función
`bc_sched`	Máxima	Prepara las transmisiones del bus y vigila el ciclo
`bc_dist`	Alta	Distribuye los datos recibidos por el bus
(varias)	Media	Sin relación con el bus
`ASI/MET`	Baja	Recolecta datos meteorológicos

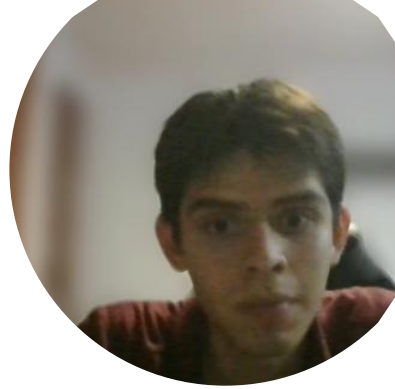
VxWorks planifica los hilos por prioridades con apropiación.



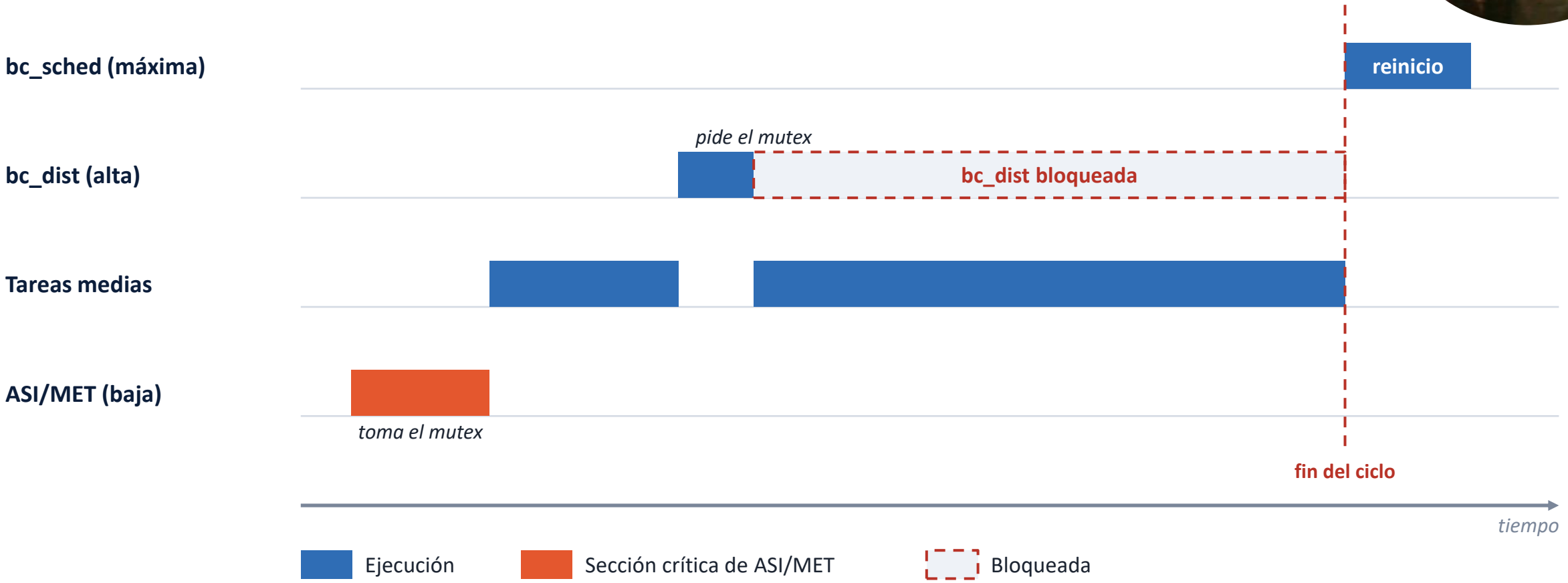
0.125 s

duración de cada ciclo del bus: plazo de tiempo real duro

bc_dist y ASI/MET comparten un mutex que estaba dentro de la función `select()` del propio VxWorks, no en el código de la misión.



Qué pasó dentro de un ciclo del bus



Al iniciar el siguiente ciclo, bc_sched ve que bc_dist no terminó, declara un error grave y reinicia la computadora.

Diagnóstico y corrección a distancia

1. bc_sched guarda el registro de eventos de VxWorks al detectar el error
2. El JPL reproduce la falla en una réplica en Tierra en menos de 18 horas
3. Se activa la herencia de prioridad en el mutex de select() mediante variables globales
4. El parche se envía y se aplica con el mecanismo de actualización de la nave

Versión popular vs. relato de Reeves

Se suele decir	Según Reeves (1997)
Lo reinició un watchdog	Lo reinició bc_sched
Se parchó con el intérprete de VxWorks	Con el mecanismo propio de la nave, que validaba los cambios
Las herramientas estaban activas por suerte	Estaban ahí por diseño



Barrido de memoria: el software completa al hardware

Tomar el siguiente bloque de 512 KB

↓

Leer el bloque completo

↓

¿El ECC detectó un error?

↓

Interrupción: corregir en memoria y registrar

↓

Repetir el barrido completo cada 10 min

Si un bloque tenía errores, se releía hasta 12 veces

493

errores por radiación corregidos

87 días

del 4 de mayo al 29 de julio de 2006



0.004 %

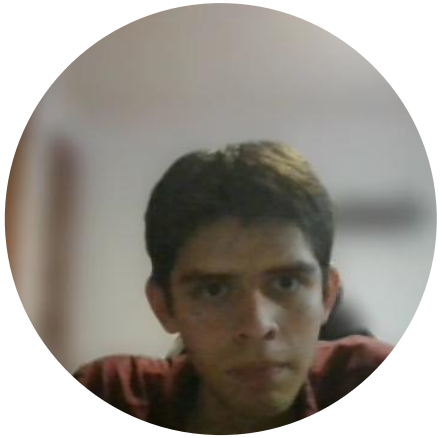
del tiempo del procesador

128

bloques de memoria recorridos

ARGOS (1999): ECC implementado en software, sin hardware adicional; la técnica EDDI detectó más del 98 % de los errores.

Un recurso que se agotaba



Índice de la memoria FLASH

~~img_0412.dat~~	*borrado*
~~log_sol15.txt~~	*borrado*
img_0413.dat	
~~tel_0088.bin~~	*borrado*
~~img_0414.dat~~	*borrado*
log_sol17.txt	
…	

Al arrancar, todo el índice se copiaba a la RAM, incluidos los borrados

Dos errores de configuración

El espacio de RAM del sistema de archivos podía crecer sin límite

Sin memoria disponible, la tarea se suspendía sin reportar error

Sol 18 · 21 de enero de 2004
Se habían creado muchos archivos: el índice agotó la memoria.

Cuando reiniciar no basta



Arranque del sistema → Montar la memoria FLASH → Copiar el índice a la RAM → Se agota la memoria → Tarea suspendida sin aviso → El monitor de salud fuerza un reinicio → *y otra vez* → Arranque del sistema

Además, la tarea quedó suspendida dentro de una sección crítica: nunca liberó un semáforo y se bloqueó hasta el apagado nocturno.

Modo crippled

- Orden procesada por hardware: arrancar sin la FLASH
- Sistema de archivos temporal en RAM, 10 veces más pequeño
- Control recuperado en el sol 21

Reiniciar solo sirve si elimina la causa del problema

Linux en Marte, pero no al mando del vuelo

Computadora de navegación
Snapdragon 801 con Linux · visión a 30 imágenes/s, telemetría

Tiempo real suave

Computadora de vuelo
Dos TI Hercules sin sistema operativo · motores 500 veces/s

Tiempo real duro

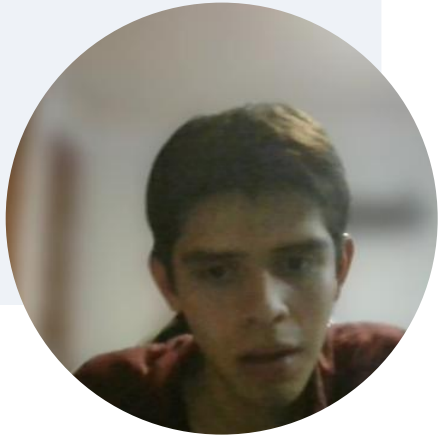
FPGA resistente a la radiación
Conecta sensores y motores, supervisa e incluye el watchdog

Supervisión

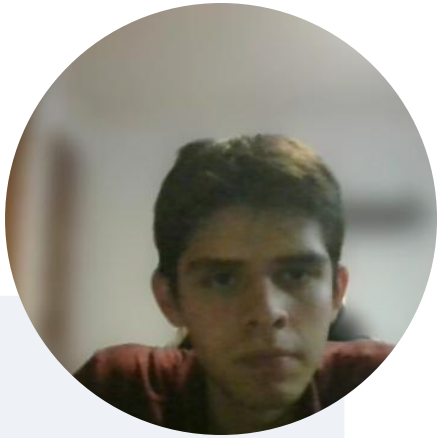
Linux solo donde un retraso no es catastrófico

Cómo se configuró Linux

- Kernel 3.4 sin parche de tiempo real
- Un núcleo por tipo de trabajo con taskset
- Software de vuelo basado en F'
- Seis copias del programa con suma de verificación



Dos incidentes



Antes del primer vuelo · 9 abr 2021

El watchdog de la FPGA detuvo una prueba: la comunicación entre los Hercules y la FPGA incumplía tiempos durante el arranque.

85 % de éxito con la nueva secuencia de arranque; corrección definitiva en el vuelo 8

No fue una falla de Linux

Vuelo 6 · 22 may 2021

A los 54 s se perdió una imagen y todas las siguientes llegaron con marca de tiempo equivocada.

> 20° de inclinación en las oscilaciones; aterrizó a unos 5 m del punto planeado

Problema de productor y consumidor

CONCLUSIONES

Diseñado para fallar bien



Cuando no puede fallar

Se apoya en la previsibilidad: prioridades con apropiación, sincronización correcta y tareas de mantenimiento como el barrido de memoria.

Cuando falla de todos modos

Detecta el problema, vuelve a un estado seguro y conserva lo esencial. Si reiniciar no basta, arranca sin el componente dañado.

Un buen sistema operativo espacial no es el que nunca falla, sino el que está diseñado para fallar bien.

Exclusión mutua, planificación, estados de procesos, memoria y semáforos: lo que vemos en clase decide el éxito de una misión.

Fuentes principales

Eyles, D. (2004). Tales from the lunar module guidance computer. 27th Annual AAS Guidance and Control Conference (AAS 04-064).

Reeves, G. E. (1997). What really happened on Mars? Authoritative account. Jet Propulsion Laboratory.

Sha, L., Rajkumar, R., & Lehoczky, J. P. (1990). Priority inheritance protocols. IEEE Transactions on Computers, 39(9), 1175-1185. https://doi.org/10.1109/12.57058

NASA. (2008). Non radiation hardened microprocessors in space based remote sensing systems (NTRS 20080014189).

Canham, T. K. (2022). The Mars Ingenuity helicopter: A victory for open-source software. 2022 IEEE Aerospace Conference. https://doi.org/10.1109/AERO53065.2022.9843438

Grip, H. F. (2021, 27 de mayo). Surviving an in-flight anomaly: What happened on Ingenuity's sixth flight. NASA Science.

McComas, D. (2012). NASA/GSFC's flight software Core Flight System. NASA NTRS 20130013412.

Bocchino, R. L., Jr., et al. (2018). F Prime: An open-source framework for small-scale flight software systems. 32nd Annual AIAA/USU Conference on Small Satellites.



La lista completa (40 referencias) está en el documento de investigación.



Gracias por su atención

Sistemas operativos en misiones espaciales

Daniel Berumen Valdez

Sistemas Operativos · Grupo 7 · Semestre 2027-1